OS Tutor — Design Document

Operating Systems Tutoring System · Fine-tuned Qwen3-4B-Instruct · 21 September 2026

Divyakush Punjabi 24BCB0126

OS Tutor is a fine-tuned Qwen3-4B-Instruct model that teaches operating systems and solves OS numerical problems exactly. It scores **100% (57/57)** on a hand-written problem suite by delegating every calculation to verified simulators while the model does the explaining — up from 63.2% when the model answers alone.

1. Problem Statement

General-purpose language models know the rules of operating systems but cannot reliably execute them, which makes them unsafe as tutors for a subject taught through worked numerical problems.

An OS course assesses students on procedures: CPU scheduling, page replacement, disk head movement, the Banker's algorithm, address translation, free-space management. Each is a rule applied over many sequential steps. A model that states Round Robin correctly and then produces the wrong Gantt chart teaches a student to trust a wrong answer.

The failure is architectural, not a gap in knowledge. Three findings frame it:

Finding	Evidence
Rule knowledge and rule execution are separate abilities	Models that state a rule correctly fail to apply it; frontier-scale models score near 50% on execution
Sequential reasoning has a deterministic horizon	Accuracy collapses after roughly 19–31 dependent steps, and the horizon grows only with the square root of model width
Self-verification does not close the gap	A model checking its own arithmetic is unreliable; correctness requires an external verifier

Our own measurements reproduced this exactly. On 57 hand-written OS problems, every one of the 7 scheduling failures contained **perfect arithmetic paired with the wrong schedule**. Scheduling also had the most training data of any category (1,733 rows) and the worst accuracy (42%), while disk scheduling had 990 rows and 100%. More data on the weak category did not fix it, because the weakness was never a data problem.

Scaling the model is not an available fix either. A larger model buys sequential depth proportional to the square root of its width, so a 14B model would extend the horizon by only about 1.4×, while training it is constrained by the free-tier hardware this project is built on.

2. Aim

Build an offline, locally-served operating-systems tutor that is numerically exact, pedagogically honest, and trainable entirely on free hardware. Four commitments define the system:

1. **Exactness on procedure.** Every answer involving a computable quantity is produced by a deterministic simulator, not by the model's arithmetic. Target: 100% on the procedural suite.
2. **Socratic integrity.** A student who shows their own attempt is taught, not handed the answer. This is enforced at the routing layer rather than requested in a prompt.
3. **Resistance to catastrophic forgetting.** Fine-tuning on a narrow OS distribution must not degrade the model's general ability, and the claim must be measured with paired item-level flips rather than asserted from a mean.
4. **Free-tier reproducibility.** All training runs on Kaggle's free GPU allocation; all inference runs on a single 8 GB laptop GPU. No paid compute anywhere in the pipeline.

The system is explicitly **not** a retrieval-augmented chatbot over a textbook. Retrieval returns text that resembles an answer; it cannot certify that a Gantt chart is correct. The aim is verified computation, which is a stronger guarantee than relevant retrieval.

3. Model Architecture

A fine-tuned 4B model handles language and pedagogy; deterministic simulators handle every number. The two are joined by an extraction step that converts a question into typed parameters.

3.1 Request flow

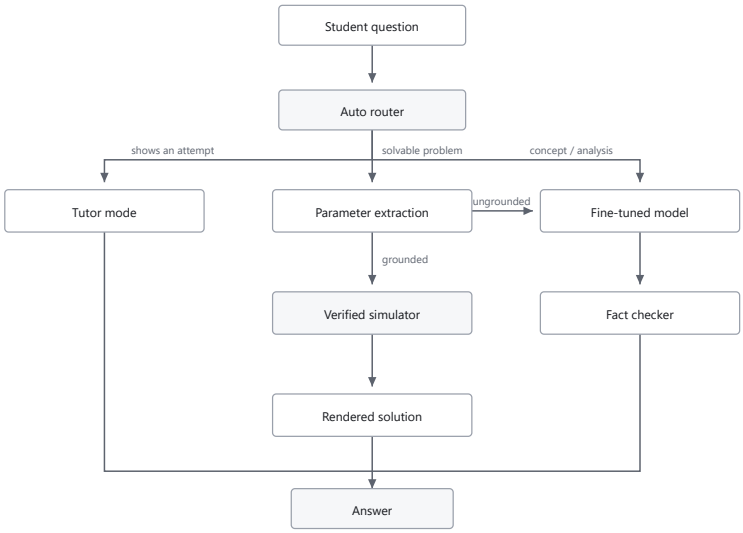


Figure 1 — Request flow. Shaded nodes are deterministic; the model is never asked to compute.

The router reads the question first. A shown attempt outranks every other signal and routes to tutoring, because handing a computed answer to a student who has already worked the problem defeats the purpose of the tutor.

3.2 Base model and adapter

Component	Choice	Reason
Base	Qwen3-4B-Instruct-2507	Non-thinking variant; reasoning appears as visible steps a student can follow

Component	Choice	Reason
Method	QLoRA via Unsloth	4-bit base keeps a 16k-token sequence inside 16 GB
LoRA rank / alpha	32 / 64	Capacity for seven procedural families without full-weight drift
Dropout, LR, schedule	0.05, 1e-4, cosine, 1 epoch	One epoch limits memorisation of the training traces
Regularisation	NEFTune α = 5	Embedding noise; improves instruction quality at no inference cost
Optimiser / precision	adamw_8bit, fp16	T4 is Turing, so bf16 is unavailable
Sequence length	16,384 tokens	Long worked solutions must fit whole, uncut
Loss masking	train_on_responses_only	Gradient flows through answers, not prompts
Sampler	Sequential	Preserves the rectifier ordering built into the dataset

3.3 Verified-solver layer

The solver is the component that turns a 63% model into a 100% system. It runs in three stages:

1. **Extract.** The question is parsed into a typed JSON parameter set across seven solver kinds, plus categories for formula, concept, analysis, shell, incomplete and off-topic. Extraction runs on the *untuned* base model, measured at 84% against the adapter's 79% — the adapter is biased toward solving rather than reading.
2. **Ground.** Every extracted number must appear in the question. Invented parameters are rejected and the item falls back to the model. A question missing a required input (frames, quantum) returns a clarifying question instead of a guess.
3. **Solve and render.** A deterministic simulator computes the result, and the answer is written back in the same worked-steps format the model was trained to produce, so the student sees no seam.

3.4 Serving stack

Layer	Component	Port
Weights	llama.cpp, Q8_0 GGUF base + f16 LoRA applied at runtime	8081
Logic	FastAPI gateway: routing, solver delegation, fact checker, loop guard	8090
Interface	Open WebUI	3000

Six modes are exposed: `ostutor` (auto-routing, default), `solve`, `analyze`, `explain`, `tutor` and `shell`. The untuned base is deliberately not selectable; it is still invoked internally at LoRA scale 0 for parameter extraction.

4. Software Requirements

Three separate stacks: one for training on Kaggle, one for serving locally, one for evaluation. Nothing in the pipeline requires a paid API or an internet connection at inference time.

Stack	Components
Training (Kaggle)	Python 3.11, PyTorch + CUDA 12.x, Unsloth, PEFT, TRL, bitsandbytes, `torchrun` for 2-GPU DDP
Serving (local)	llama.cpp build b11050 (CUDA), FastAPI + uvicorn + httpx gateway, Open WebUI 0.11.3
Evaluation	transformers + PEFT for logit scoring, custom simulator-exact grader, MMLU subject sets
Data pipeline	Pure Python: cleaning, deduplication, decontamination, quality scoring, gate checks
Platform	Windows 11, CUDA 12.x driver; the pipeline itself is OS-independent

The base model is Qwen3-4B-Instruct-2507 under Apache 2.0. Every dataset admitted to training carries a licence permitting commercial use; sources with unclear or non-commercial terms are used only as held-out test oracles and are never redistributed.

5. Hardware Requirements

Training and inference run on entirely separate machines, both free or already owned.

5.1 Training — Kaggle free tier

Item	Specification
GPU	2 × NVIDIA Tesla T4, 16 GB each
Parallelism	DDP via `torchrun --nproc_per_node=2`
Precision constraint	Turing architecture — fp16 only, no bf16
Session limit	12 hours per run, 30 GPU-hours per week

Because DDP replicates the full model on each card, the second GPU adds throughput but no memory headroom. The 16,384-token sequence length is what sets the real memory ceiling, and it is why a 4B base was selected over a larger one.

5.2 Inference and evaluation — local workstation

Item	Specification	Measured usage
GPU	RTX 4060 Laptop, 8,188 MiB VRAM	7.5 GB serving at Q8_0 with 20,480-token context
System RAM	15.6 GB DDR5-5200, single DIMM	Gateway + WebUI + model server
PCIe	Gen 4 ×8, ~13 GB/s effective	Not a bottleneck at this model size
Storage	512 GB NVMe SSD	~4.3 GB base GGUF + 132 MB adapter

5.3 Minimum to reproduce

Inference needs a 6 GB GPU at Q4_K_M quantisation, or CPU-only with 8 GB of RAM at reduced speed. Training needs any 16 GB accelerator; the Kaggle configuration is the free option, not a requirement.

6. What Makes This Pipeline Unique

Four design decisions separate this system from a fine-tuned chatbot or a textbook RAG assistant. Each was adopted to fix a specific measured failure.

6.1 Verified computation instead of retrieval

A RAG tutor retrieves passages that resemble an answer. It cannot certify that a Gantt chart is right, because the correct chart for a student's specific numbers exists in no document. This system replaces retrieval with **execution**: deterministic simulators compute ground truth for the exact parameters asked about.

This is the single highest-impact decision in the project. It lifted the procedural suite from 63.2% to **100% (57/57)** with **no retraining**, and cut a full evaluation pass from 76 minutes to about 150 seconds.

The delegation is also *safe by construction*. Extraction is grounded — every parameter must appear in the question — so the solver cannot answer a question it misread. Ungrounded extractions fall back to the model rather than producing a confident fabrication.

6.2 Layered anti-forgetting design

Rather than one technique, the dataset and training configuration combine several, each addressing a different mechanism:

Technique	Purpose
16% replay data on a spaced schedule	Keeps general-domain gradients present throughout training
Surprise-ranked replay	Prioritises items the model is most at risk of losing
Rectifier interleaving	Orders categories so later skills reinforce earlier ones instead of overwriting them
Self-distillation (SDFT)	Trains on the model's own re-expressed outputs, reducing distribution shift
LoRA with a single epoch	Low-rank updates constrain drift; one epoch limits memorisation

Section 7.3 reports the measured result, including item-level flips.

6.3 Socratic integrity enforced structurally

Most tutoring systems ask the model in a prompt not to give away answers, which fails under pressure. Here it is a property of the architecture:

- A shown attempt is detected lexically, and that signal **outranks the intent classifier**.
- Tutoring mode is denied access to the solver *and* to the fact checker, so there is no path by which a computed answer can reach a student who is still working.
- Routing is tested as a component in its own right: **28/28** on labelled cases, with zero misroutes on shown-attempt items.

6.4 Auditable, reproducible dataset construction

The training set is 11,760 rows built by a gated pipeline: per-category cleaning, quality scoring, deduplication, and decontamination against every evaluation set. Nineteen automated gates must pass before a build is

accepted, and the build is content-addressed to a reproducible fingerprint (`f56a7113e4aa5c0e`). A failed gate blocks the build rather than emitting a warning.

7. Outcome (Expected)

At completion the system is expected to hold 100% on procedural problems, raise OS theory accuracy to 90% or above, and retain general ability within 1 percentage point of the base model. The table below states the expected outcome for every category alongside the value measured so far, so that projection and result are never confused.

7.1 Expected outcome at completion

Category	Measured to date	Expected at completion	Basis for the expectation
Procedural accuracy (OS numericals)	100% (57/57)	**100%** on an expanded 150-item suite	Simulators are deterministic, so accuracy does not decay with suite size; extending delegation to the 11 formula kinds removes the remaining model-answered items
Model-only accuracy (no solver)	63.2%	**70–75%**	Restoring 6 gold explanations dropped during cleaning, plus additional worked-solution rows for scheduling, the weakest family
OS theory MCQ	85.71% (n = 70)	≥ **90%** on a 200-item set	Targeted declarative replay for the regressed items; a 200-item set replaces an estimate currently resting on 70 questions
General ability retained	79.50% (−0.75 pp)	**Within 1 pp** of base	Already achieved; 16% spaced replay holds it without further work
Adjacent CS transfer	67.33% (+2.66 pp)	≥ **67%**	Already achieved; no further action required
Routing accuracy	100% (28/28)	≥ **95%** on 100 labelled cases	A larger labelled set will contain harder cases than the current 28
Conceptual tutoring quality	Not yet measured	≥ **80%** rubric-scored	Requires an evaluation set that does not yet exist; this is the largest open item
Training cost	2 × T4, free tier	**Unchanged**	No step in the remaining plan requires paid compute

Values in the “Measured to date” column are results obtained on 21 September 2026 and are reported in full in sections 7.2–7.5. Values in the “Expected at completion” column are targets, not results.

7.2 Procedural suite, by question type

Measured on 57 hand-written problems, graded by simulator-exact comparison.

Question type	Model alone	With verified solver
Scheduling	5/12	**12/12**
Paging	5/8	**8/8**

Question type	Model alone	With verified solver
Disk scheduling	4/7	**7/7**
Banker's safety	3/4	**4/4**
Free-space management	0/4	**4/4**
Address translation	1/3	**3/3**
Clarification required	3/3	**3/3**
Total	**36/57 (63.2%)**	**57/57 (100%)**

7.3 Catastrophic forgetting

Paired item-level comparison, 770 identical items scored before and after fine-tuning. Flips are reported because an unchanged mean can hide large cancelling changes.

Benchmark	Base	Fine-tuned	Change	Right→wrong	Wrong→right
General MMLU (400 items)	80.25%	79.50%	−0.75 pp	13	10
Adjacent CS (300 items)	64.67%	**67.33%**	**+2.66 pp**	11	19
OS theory (70 items)	90.00%	85.71%	−4.29 pp	4	1

No catastrophic forgetting occurred. General ability is intact: 13 items lost against 10 gained across 400 is within scoring noise. Adjacent computer science *improved*, and the flip counts confirm genuine transfer rather than a favourable average.

One result runs against expectation: **OS declarative theory measured 4.29 points below the base model**, losing four items and gaining one. Procedural skill rose sharply while factual recall in the same domain dipped — the opposite of the general-knowledge loss that was anticipated. The difference is four questions out of seventy, and the base model was already near ceiling at 90%, so the current figure is a signal to investigate rather than a settled result.

Expected outcome: ≥ 90% on a 200-item theory set. Two changes support this. A set of 200 questions replaces an estimate that presently turns on four items, and targeted declarative replay reinforces the factual material that heavy procedural training displaced. Until that larger evaluation exists, the 85.71% figure stands as measured.

7.4 Training behaviour

Validation loss fell monotonically from 0.4413 to 0.2713 across one epoch, with final training loss 0.2710 against validation 0.2754. The near-identical values indicate the model fitted the task without memorising the training traces.

7.5 System verification

Suite	Result
Data pipeline tests	89 checks, all passed
Serving gateway tests	25 checks, all passed

Suite	Result
Router classification	28/28
Dataset build gates	19/19

7.6 Work remaining to reach the expected outcome

Task	Target it serves	Effort
Expand the OS theory MCQ set from 70 to 200 items	OS theory ≥ 90%	Authoring, then one evaluation pass
Add targeted declarative replay for the regressed topics	OS theory ≥ 90%	One dataset rebuild and retrain
Extend solver delegation to the 11 formula kinds	Procedural 100% at larger scale	Simulator work only, no retraining
Build a rubric-scored conceptual tutoring evaluation	Tutoring quality ≥ 80%	Largest open item; no set exists yet
Measure base and v2.2 on the procedural suite	Completes the comparison	~45 minutes of GPU time
Grow the routing test set from 28 to 100 labelled cases	Routing ≥ 95%	Labelling only

7.7 Known limitations

- The OS theory result in 7.3 rests on 70 items and is not yet explained; the expected recovery to 90% depends on a larger set that has still to be written.
- Base and previous-version comparisons on the procedural suite are outstanding. An earlier measurement was invalidated by a harness fault and was withdrawn rather than reported.
- Solver delegation covers seven procedural families. Formula-style questions are answered by the model directly and carry its unaided accuracy.
- The conceptual half of tutoring has no external verifier. It is the weakest area of the system and the one with no measurement at all.